

LLM - PROYECTO 25-26

Aplicación Web Completa con Frontend, Backend y Base de Datos

Documentación Técnica del Proyecto

Autores:

Jairo Prats Albal
Gabriela Marinova Todorova
Aarón Martínez López

Curso: 1º DAW • Año Académico: 2025-2026

1. Índice de Contenidos

Este documento recoge la documentación técnica completa del proyecto LLM 25-26, una aplicación web de comercio electrónico inspirada en la tienda de Nintendo.

1. Índice de Contenidos.....	2
2. Descripción del Proyecto.....	3
2.1 Motivación.....	3
2.2 Objetivo Principal.....	3
2.3 Necesidades que Cubre.....	3
3. Análisis de Requisitos.....	4
3.1 Requisitos Funcionales.....	4
3.2 Requisitos No Funcionales.....	4
4. Esquema de Contenidos (Wireframes / Mockup).....	5
4.1 Estructura General de la Web.....	5
4.2 Estructura del Wireframe Inicial.....	5
4.3 Descripción Visual de las Pantallas Principales.....	5
5. Tecnologías Frontend y Backend.....	6
5.1 Frontend.....	6
5.2 Backend.....	6
5.3 Arquitectura General.....	7
6. Diseño de la Persistencia de Datos.....	8
6.1 Base de Datos MySQL.....	8
6.2 Persistencia en el Frontend (localStorage).....	8
6.3 Sincronización del Carrito del Invitado.....	9
7. Pasos Principales del Desarrollo.....	10
7.1 Configuración del Entorno y Paquetes Utilizados.....	10
7.2 Definición e Implementación de Modelos, Controladores y Rutas.....	10
7.3 Componentes de la Vista y sus Funcionalidades.....	10
8. Conclusión.....	12
8.1 Problemas Encontrados y sus Soluciones.....	12
8.2 Aportaciones a Nivel Personal.....	12
8.3 Mejoras que Añadiríamos.....	13

2. Descripción del Proyecto

2.1 Motivación

La motivación principal de este proyecto es afianzar los conocimientos de desarrollo web full-stack y crear una réplica funcional de una tienda en línea real, concretamente inspirada en la tienda oficial de Nintendo. El proyecto sirve para demostrar las habilidades adquiridas a lo largo del curso y consolidar el flujo de trabajo completo: desde el diseño de la base de datos hasta la interfaz de usuario.

Los miembros del equipo han colaborado dividiéndose las responsabilidades de manera equitativa, aportando cada uno su conocimiento en áreas específicas del desarrollo.

2.2 Objetivo Principal

Implementar una tienda en línea con funcionalidades completas que incluyan:

- Catálogo de productos con información detallada (juegos para Nintendo Switch y Switch 2)
- Carrito de compras con persistencia local mediante localStorage
- Lista de deseos / favoritos para usuarios registrados
- Panel de administración (Back Office) para gestionar todo el catálogo
- Sistema de búsqueda, filtrado y ordenación de productos
- Autenticación de usuarios con roles diferenciados (usuario normal y administrador)
- Soporte multi-idioma mediante i18n

2.3 Necesidades que Cubre

La aplicación cubre las siguientes necesidades para dos tipos de usuarios:

Usuario Final (Cliente)

- Explorar el catálogo completo de juegos disponibles por plataforma
- Buscar productos por título y filtrar por plataforma o precio
- Añadir productos al carrito y gestionar la compra
- Guardar juegos favoritos en la lista de deseos
- Crear y gestionar su cuenta personal
- Aplicar códigos de descuento (codigokonami, retro)

Administrador

- Gestionar el catálogo de juegos (crear, editar, eliminar)
- Administrar usuarios (ver, modificar estado, eliminar)
- Gestionar noticias, novedades y carrusel de la página principal
- Controlar plataformas, etiquetas y aplicaciones

3. Análisis de Requisitos

3.1 Requisitos Funcionales

Front Office (Zona Pública)

- 01 - Página de inicio con carrusel de anuncios y secciones de novedades
- 02 - Catálogo de juegos con paginación (8 juegos por página)
- 03 - Búsqueda de productos en tiempo real por título
- 04 - Filtrado por plataforma (Nintendo Switch / Nintendo Switch 2)
- 05 - Ordenación por precio (ascendente / descendente / fecha)
- 06 - Añadir productos al carrito desde el catálogo
- 07 - Carrito de compras con cálculo de subtotal, impuestos (21%) y total
- 08 - Aplicación de códigos de descuento en el carrito
- 09 - Lista de deseos / favoritos persistente
- 10 - Registro y autenticación de usuarios con validación de contraseña
- 11 - Selección de idioma en la página de login (ES, EN, ZH, JA, KO, PT)
- 12 - Gestión del perfil de usuario (nombre, email, contraseña, avatar)

Back Office (Zona Privada - Administrador)

- 13 - Panel unificado de administración accesible por rol admin/content_editor/game_manager
- 14 - CRUD completo de usuarios con campo de búsqueda
- 15 - CRUD completo de juegos con asignación de plataforma y etiquetas
- 16 - Gestión del carrusel de imágenes de la página principal
- 17 - Gestión de noticias, novedades principales y novedades secundarias
- 18 - Control de estado de usuarios (activo, inactivo, bloqueado)
- 19 - Indicador de estado de la API en tiempo real

3.2 Requisitos No Funcionales

- 01 - Arquitectura cliente-servidor con API RESTful
- 02 - Persistencia del carrito para usuarios no registrados vía localStorage
- 03 - Sincronización del carrito local con la base de datos al iniciar sesión
- 04 - Gestión de CORS para permitir peticiones cross-origin
- 05 - Respuestas de la API siempre en formato JSON
- 06 - Contraseñas almacenadas con hash seguro
- 07 - Control de acceso basado en roles (RBAC)
- 08 - Diseño visual inspirado en la página oficial de Nintendo (rojo y blanco)

4. Esquema de Contenidos (Wireframes / Mockup)

4.1 Estructura General de la Web

El proyecto está dividido en dos zonas principales:

Front Office (Zona Pública)

- index.html — Página de inicio con carrusel y secciones de novedades/juegos
- juegos.html — Catálogo de juegos con filtros y paginación
- carrito.html — Carrito de compras con resumen del pedido
- wishlist.html — Lista de juegos favoritos
- login.html — Autenticación de usuarios con selección de idioma
- register.html — Creación de nueva cuenta
- count.html — Panel de gestión de la cuenta personal
- etc..

Back Office (Zona Privada)

- admin.html — Panel unificado de gestión (usuarios, juegos, noticias, carrusel, etc.)

4.2 Estructura del Wireframe Inicial

El wireframe inicial definía para la página de inicio (index.html) las siguientes secciones:

1. Primera sección: Carrusel horizontal con anuncios e imágenes destacadas
2. Segunda sección: Novedades con un post destacado y cuatro miniaturas
3. Tercera sección: Juegos con sub-pestañas (Destacados, Próximamente, Lanzamientos recientes)
4. Cuarta sección: Aplicaciones para dispositivos inteligentes
5. Quinta sección: Hardware y accesorios
6. Sexta sección: Juegos disponibles para Switch 1 y Switch 2

4.3 Descripción Visual de las Pantallas Principales

A continuación se describe el diseño final de las pantallas más relevantes:

Página de Inicio (index.html)

La barra superior incluye cuatro iconos: búsqueda, lista de deseos, carrito y cuenta de usuario. Al iniciar sesión como administrador, aparece también el acceso al panel de Admin.

Catálogo de Juegos (juegos.html)

Muestra todos los juegos en una cuadrícula de 4 columnas. Incluye un campo de búsqueda instantánea, un desplegable de plataforma y uno de ordenación. La paginación se genera automáticamente a partir de 8 juegos.

Carrito (carrito.html)

Muestra los productos añadidos, el subtotal, los impuestos del 21% y el total. Permite eliminar productos y aplicar códigos de descuento. El botón 'Continuar a la caja' lleva al proceso de pago.

Back Office (admin.html)

Panel lateral izquierdo con menú de secciones (Usuarios, Juegos, Aplicaciones, Carrusel, Tienda, Novedades, Noticias). La zona derecha muestra la tabla de gestión con botones de edición y eliminación para cada registro, y el botón 'Añadir Nuevo' para crear entradas.

5. Tecnologías Frontend y Backend

5.1 Frontend

Las tecnologías escogidas para el lado cliente son las siguientes:

Tecnología	Uso / Función	Justificación
HTML5	Estructura semántica de todas las páginas	Estándar web universal, compatible con todos los navegadores. No requiere compilación ni herramientas adicionales.
CSS	Estilización y diseño visual	Permite un control completo sobre el diseño. Se ha evitado depender de frameworks externos para aprender los fundamentos.
JavaScript	Lógica de la interfaz, peticiones AJAX, manipulación del DOM	Lenguaje nativo del navegador. El uso de fetch API y async/await ofrece código moderno y legible.
localStorage	Persistencia del carrito y wishlist para usuarios no registrados	API nativa del navegador, sin dependencias. Permite guardar datos entre sesiones sin necesidad de login.
Fetch API	Comunicación asincrónica con el backend	Nativa en navegadores modernos. Más limpia que XMLHttpRequest y sin necesidad de jQuery.
i18n (custom)	Internacionalización de la página de login	Permite adaptar la aplicación a diferentes mercados (ES, EN, ZH, JA, KO, PT), replicando el comportamiento de Nintendo.

5.2 Backend

Las tecnologías escogidas para el lado servidor son las siguientes:

Tecnología	Uso / Función	Justificación
PHP	Servidor web y lógica de negocio	Ligero, de fácil despliegue y muy adecuado para proyectos de curso. No requiere configuración compleja.
PDO (PHP Data Objects)	Abstracción de acceso a la base de datos	Ofrece consultas preparadas (prepared statements) que previenen inyecciones SQL. Capa de abstracción estándar de PHP.
MySQL	Base de datos relacional	Perfectamente integrado con PHP. La licencia gratuita facilita su uso en entornos educativos.

API RESTful	Arquitectura de la capa de comunicació	Separación clara entre frontend y backend. Permite escalar o cambiar el cliente sin modificar el servidor.
CORS	Gestión de peticiones cross-origin	Necesario para permitir que el frontend (diferente puerto o dominio) pueda comunicarse con el backend de forma segura.

5.3 Arquitectura General

La arquitectura sigue el patrón cliente-servidor con una API RESTful como capa intermedia:

- router.php — Punto de entrada único del backend. Enruta las peticiones hacia api/index.php para los endpoints de la API, o hacia /public para los archivos estáticos.
- api/index.php — Conecta PDO con MySQL y gestiona todos los endpoints de la API.
- Endpoints disponibles: /usuarios, /juegos, /carrito, /favoritos, /noticias, /novedades, /carousel, /plataformas
- Verbos HTTP: GET (listar/obtener), POST (crear), PUT (actualizar), DELETE (eliminar). Siempre responden en JSON.

6. Diseño de la Persistencia de Datos

6.1 Base de Datos MySQL

La base de datos está formada por 12 tablas relacionales que cubren todas las necesidades de la aplicación:

Tablas Principales

- usuarios — Usuarios del sistema. Campos: id, nombre, email, password (hash), rol (user/admin/content_editor/game_manager), estado (activo/inactivo/bloqueado), fecha_registro, avatar, debe_cambiar_password.
- juegos — Catálogo de juegos. Campos: id, titulo, plataforma_id (FK), fecha_lanzamiento, imagen, es_nueva_consola, precio.
- plataformas — Plataformas disponibles (Nintendo Switch, Nintendo Switch 2). Campos: id, nombre, tipo.
- carrito — Ítems en el carrito de compras. Campos: id, usuario_id (FK), juego_id (FK), cantidad, fecha_agregado.
- favoritos — Lista de deseos. Campos: id, usuario_id (FK), juego_id (FK), fecha_agregado.

Tablas de Contenido Editorial

- noticias — Noticias de la página de inicio. Campos: id, titulo, imagen, etiqueta_id (FK), fecha, link.
- novedades_principal — Noticia/novedad destacada de la sección novedades. Campos: id, imagen, etiqueta_id (FK), titulo, descripcion, created_at, updated_at.
- novedades_secundarias — Noticias secundarias. Campos: id, orden, imagen, etiqueta_id (FK), titulo, created_at, updated_at.
- carousel — Imágenes del carrusel principal. Campos: id, url_imagen, alt, titulo, boton_texto, link, orden.
- etiquetas — diferenciación entre secciones de la página de inicio. Campos: id, nombre.
- aplicaciones — Aplicaciones para dispositivos inteligentes. Campos: id, nombre, plataforma_id (FK), fecha_lanzamiento, imagen.
- mynintendostore — Ítems para la sección MyNintendo Store. Campos: id, nombre, descripcion, imagen.

6.2 Persistencia en el Frontend (localStorage)

Para los usuarios no autenticados, la aplicación utiliza localStorage para mantener el estado entre sesiones:

- loggedUser[] — Datos de la sesión activa (nombre, email, rol, token). Se elimina al cerrar la sesión.
- carrito[] — Array de juegos añadidos al carrito por usuarios no registrados (invitados). Se sincroniza con la BD al iniciar sesión.
- wishlist[] — Array de juegos añadidos a la lista de deseos por invitados.

6.3 Sincronización del Carrito del Invitado

El flujo de login está diseñado para no perder los datos del carrito de un usuario no registrado cuando se autentica:

7. El frontend hace GET /api/usuarios para obtener todos los usuarios.
8. JavaScript busca el email + contraseña en el array del cliente.
9. Se guarda el usuario autenticado en `loggedUser[]` en `localStorage`.
10. Se sincroniza el carrito[] local hacia la base de datos con llamadas PUT.

7. Pasos Principales del Desarrollo

7.1 Configuración del Entorno y Paquetes Utilizados

El entorno de desarrollo se ha configurado con las siguientes herramientas:

- Servidor local: PHP built-in server (php -S localhost:3000 router.php) para servir tanto la API como los archivos estáticos.
- Base de datos: MySQL con un servidor local (XAMPP) para gestionar la base de datos tienda.
- Herramientas de depuración: Consola del navegador y DevTools Network para monitorizar las peticiones HTTP.
- Control de versiones: Git + GitHub para la colaboración entre los miembros del equipo.
- Editor de código: Visual Studio Code, y Antigravity.
- Cliente de base de datos: phpMyAdmin para gestionar y visualizar la estructura de las tablas.

7.2 Definición e Implementación de Modelos, Controladores y Rutas

Diseño de la Base de Datos

La primera fase consistió en diseñar el esquema relacional de la base de datos. Se identificaron las entidades principales (juegos, usuarios, plataformas, carrito, favoritos) y las relaciones entre ellas. Las claves foráneas aseguran la integridad referencial.

Implementación del Backend (API RESTful)

Se ha implementado un único punto de entrada (router.php) que redirige todas las peticiones. El archivo api/index.php gestiona los endpoints:

- Conexión PDO a MySQL con gestión de errores
- Detección del método HTTP (GET/POST/PUT/DELETE) para ejecutar la acción correspondiente
- Retorno siempre en JSON con headers CORS configurados
- Hashing de contraseñas con password_hash() de PHP
- Validación de rol en las rutas protegidas del Back Office

7.3 Componentes de la Vista y sus Funcionalidades

index.html - Página Principal

Implementa un carrusel automático, secciones de novedades cargadas dinámicamente desde la API, y una cuadrícula de juegos destacados con sub-pestañas de filtrado. Todos los datos se obtienen vía fetch API.

juegos.html - Catálogo

Carga todos los juegos desde la API e implementa búsqueda en tiempo real, filtrado por plataforma y ordenación por precio. La paginación está calculada en el cliente y se regenera en cada filtrado. El botón de añadir al carrito actualiza la BD si el usuario está autenticado, o localStorage si es invitado.

carrito.html - Carrito

Lee los ítems del carrito (BD o localStorage) y calcula el total con impuestos. Implementa la función de aplicación de códigos de descuento: 'codigokonami' (50% de descuento global) y 'retro' (descuento en juegos de Switch 1).

login.html / register.html

Incluye validación de contraseña con expresiones regulares (mínimo 8 caracteres, una mayúscula, una minúscula, un número, sin espacios). El login sigue el flujo de sincronización descrito en la sección 6.3. La selección de idioma utiliza un módulo i18n personalizado.

admin.html - Back Office

Panel unificado con menú lateral. Detecta el rol del usuario autenticado desde localStorage y muestra solo las secciones permitidas. Las operaciones CRUD se realizan vía fetch a la API. Incluye un buscador global para filtrar registros de la sección activa, y modales para los formularios de creación y edición.

8. Conclusión

8.1 Problemas Encontrados y sus Soluciones

A lo largo del desarrollo del proyecto hemos encontrado diversas dificultades técnicas que hemos tenido que resolver:

Diseño de la Base de Datos Funcional

Problema: Determinar cuántas tablas son necesarias, qué campos deben tener y cómo relacionarlas correctamente sin redundancias.

Solución: Realizamos varias iteraciones del modelo relacional hasta llegar a las 12 tablas finales. Utilizamos phpMyAdmin para visualizar y ajustar las relaciones de forma visual.

Implementación de Expresiones Regulares para la Verificación de Contraseñas

Problema: Implementar una validación que siga los requisitos de seguridad (8+ caracteres, mayúsculas, minúsculas, números, sin espacios) tanto en el frontend como en el backend.

Solución: Se crearon expresiones regulares específicas para cada condición y se mostraron indicadores visuales en tiempo real al usuario.

Planteamiento del Back Office

Problema: Diseñar un panel de administración unificado que gestione múltiples entidades (usuarios, juegos, noticias...) sin crear una página independiente para cada una.

Solución: Se implementó un sistema de pestañas/secciones dentro de una sola página HTML, cargando dinámicamente el contenido de cada sección al hacer clic en el menú lateral.

Diferenciación Entre Usuarios Normales y Administradores

Problema: Controlar el acceso a secciones privadas (Back Office) sin usar sesiones PHP, ya que el frontend se gestiona completamente en el navegador.

Solución: El rol del usuario se almacena en localStorage al iniciar sesión, y el JavaScript comprueba este rol antes de mostrar u ocultar los elementos de administración.

Sincronización del Carrito del Invitado

Problema: Cuando un usuario invitado que ha añadido juegos al carrito (localStorage) se autentica, es necesario fusionar el carrito local con el carrito existente en la base de datos.

Solución: Se implementó un flujo de login en 4 pasos que sincroniza automáticamente el carrito local hacia la BD tras la autenticación, evitando duplicados.

8.2 Aportaciones a Nivel Personal

Este proyecto ha sido una experiencia muy enriquecedora para los tres miembros del equipo. Hemos profundizado en:

- El funcionamiento real de una API RESTful y cómo se comunica con el frontend mediante fetch.
- La importancia del diseño previo de la base de datos para evitar problemas en fases avanzadas.
- La gestión de un proyecto en equipo con Git, resolviendo conflictos y manteniendo una estructura de código ordenada.
- Cómo replicar el diseño visual de una aplicación real y adaptarlo a nuestras necesidades.

8.3 Mejoras que Añadiríamos

Si dispusiéramos de más tiempo, las mejoras que incorporaríamos al proyecto son:

- Implementar un sistema de sesiones PHP genuinas para una autenticación más segura.
- Añadir paginación en el servidor para mejorar el rendimiento con un catálogo más grande.
- Añadir la función de reseñas y puntuaciones de juegos por parte de los usuarios registrados.
- Mejorar la accesibilidad (etiquetas ARIA, navegación por teclado) siguiendo los estándares WCAG.
- Desplegar la aplicación en un entorno de producción real

Fin de la documentación

Jairo Prats Albal • Gabriela Marinova Todorova • Aarón Martínez López — 1º DAW 2025-2026